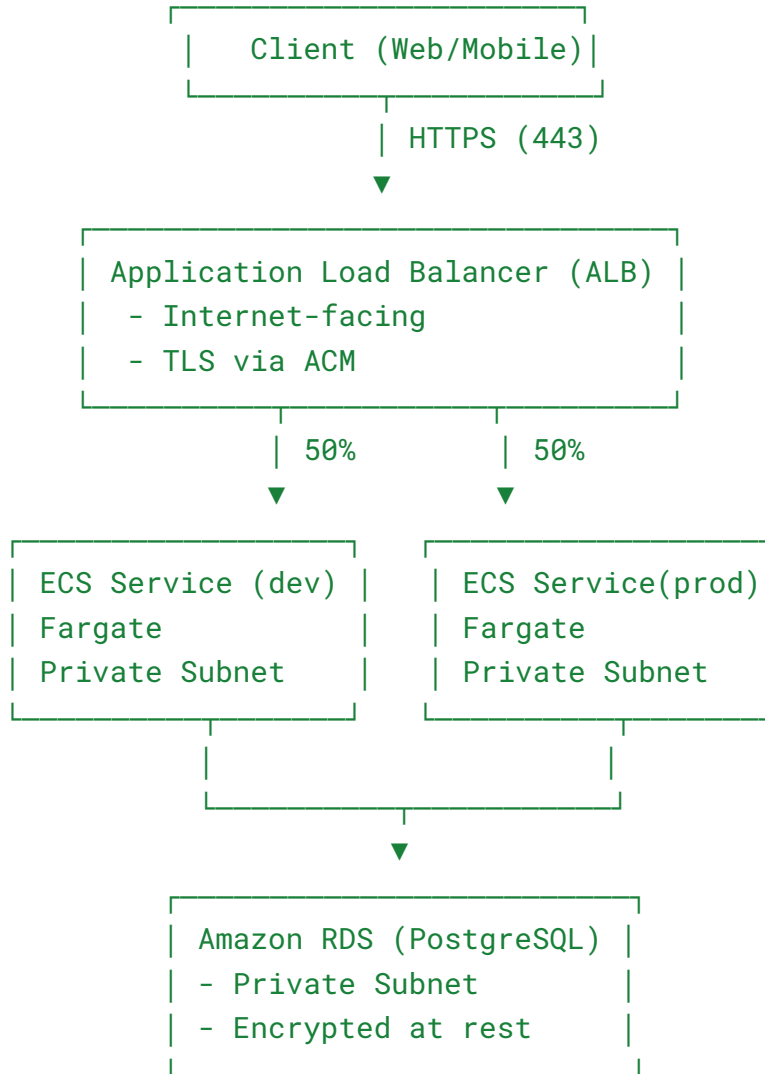




Simple Architecture Diagram





INFRASTRUCTURE DOCUMENTATION

1. Objective

Provision a secure, scalable, and production-ready AWS infrastructure using Terraform to host a containerized Node.js API, following AWS best practices.

2. Networking (Assessment: Networking)

- VPC
 - CIDR: **10.0.0.0/16**
 - Multi-AZ: **ap-south-1a, ap-south-1b**
- Subnets
 - Public subnets: ALB, NAT Gateway
 - Private subnets: ECS services
 - Database subnets: RDS
- NAT Gateway
 - Allows outbound internet access for private workloads

✓ *Meets requirement: Multi-AZ VPC with public/private/database subnets and NAT Gateway*

3. Compute (Assessment: Compute)

- Amazon ECS (Fargate)
 - Single ECS cluster
 - Two services:
 - **task-api-dev**
 - **task-api-prod**
 - Stateless container workloads
 - No EC2 management required

✓ *Meets requirement: ECS Fargate with dev/prod services in shared cluster*

4. Database (Assessment: Database)

- Amazon RDS – PostgreSQL
 - Engine: PostgreSQL 15
 - Private subnet deployment
 - Encrypted at rest
 - Automated backups enabled
 - Accessible only from ECS security group

✓ *Meets requirement: RDS PostgreSQL in private subnet with encryption and backups*

5. Load Balancing (Assessment: Load Balancing)

- **Application Load Balancer**
 - Internet-facing
 - HTTP (80) → Redirect to HTTPS
 - HTTPS (443) → Forward to target groups
 - Weighted routing:
 - 50% dev
 - 50% prod

✓ *Meets requirement: Single ALB routing to both services via round*

6. Security (Assessment: Security)

- **Security Groups**
 - ALB: Public access on 80/443
 - ECS: Accepts traffic only from ALB
 - RDS: Accepts traffic only from ECS
- **TLS**
 - AWS Certificate Manager (ACM)
 - Custom domain: **api.techaxion.com**
 - DNS validation via Cloudflare
 - ✓ *Meets requirement: Least-privilege IAM and minimal network exposure*

7. State Management (Assessment: State)

- Terraform used as Infrastructure as Code
- Centralized state
- Locking mechanism during operations

✓ *Meets requirement: Remote state with locking mechanism*

8. Cleanup

After assessment verification:

- Terraform destroy executed
- ACM certificate deleted
- DNS records removed
- Ensured zero running AWS resources



APPLICATION DOCUMENTATION

1. Application Overview

- Runtime: Node.js (Next.js API routes)
- Containerized: Docker
- Port: 3000

2. API Endpoints

Method	Endpoint	Description
GET	/api/health	Health check
GET	/api/tasks	Fetch tasks
GET	/api/tasks/[id]	Fetch task by ID

Health Response Example

```
{  
  "status": "ok",  
  "environment": "dev"  
}
```

Refreshing alternates between **dev** and **prod**, validating load balancing.

3. Dockerization Flow

Dockerfile Summary

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 3000
CMD ["npm", "run", "start"]
```

- Produces a production-ready image
 - Ensures consistent runtime across environments
-

4. Image Build & Registry (Assessment: CI/CD Style Flow)

Build Image

```
docker build -t task-api:dev .
```

Tag for ECR

```
docker tag task-api:dev \
<account-id>.dkr.ecr.ap-south-1.amazonaws.com/task-api:dev
```

Authenticate & Push

```
aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin <account-id>.dkr.ecr.ap-south-1.amazonaws.com
```

```
docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/task-api:dev
```

5. Runtime Deployment

- ECS task definitions reference ECR images
 - Environment variables injected at runtime
 - Tasks run in private subnets
 - No public IP assigned
-

6. End-to-End Request Flow

Client → HTTPS → ALB
→ ECS Fargate (dev/prod)
→ PostgreSQL (RDS)



FINAL ASSESSMENT ALIGNMENT SUMMARY

Requirement	Status
Multi-AZ networking	✓
Private database	✓
ECS Fargate	✓
Single ALB	✓
Round-robin routing	✓
HTTPS	✓
Least privilege security	✓
Terraform IaC	✓
Clean teardown	✓

Closing Note (Assessment-Style)

This implementation demonstrates practical experience in designing, securing, deploying, and validating a cloud-native application using AWS managed services and Infrastructure as Code. The architecture aligns with production standards and can be extended easily for scale, monitoring, and CI/CD automation.